

Programación de servicios y procesos
José Cruces Aparicio - Tarea 1 UD 2 2º DAM

- **TÍTULO**

Creación de hilos cooperativos.

- **SITUACIÓN**

Estás trabajando en el servicio técnico del ayuntamiento de tu ciudad como desarrollador de aplicaciones.

Tienes una lista de todas las temperaturas máximas que se han dado cada día en la ciudad donde trabajas los últimos 10 años y las tienes guardadas en un array de enteros.

El ayuntamiento quiere publicar en la web la temperatura máxima de los últimos 10 años. ¿Cómo conseguirlo?

- **INSTRUCCIONES**

Para hacerlo, debes seguir los siguientes pasos:

- Simula las temperaturas llenando un array de 3.650 posiciones y llénalo de forma aleatoria con números enteros, cogiendo valores entre -30 y 50.
- Para encontrar la temperatura más alta, divide el array en partes y crea tantos hilos como partes hayas creado. Cada hilo tendrá que buscar dentro del array asignado cuál es la temperatura más alta.
- Para finalizar, una vez encontrado el máximo de cada subconjunto del array, este te dirá cuál es el número máximo de los devueltos por cada hilo.

Codigo generado :

Clase : maxTemperatura.java

```
// Importamos la libreria java.util.Random que es la que utilizaremos para generar las temperaturas aleatorias
import java.util.Random;

//Clase principal que calcula la temperatura maxima usando varios hilos
public class maxTemperatura {

    // Clase que representa un hilo que busca el máximo en un rango del array
    static class MaxFinder extends Thread {
        // Declaramos el array que vamos a utilizar para almacenar las temperaturas de los 10 años
        private int[] array;
        // Declaramos las variables start y end
        private int start, end;
        // Declaramos la variable donde se almacenara el valor maximo encontrado por este hilo
        private int localMax;
        // Identificador del hilo
        private int idHilo;

        // Constructor que inicializa el array y los limites que procesara el hilo
        public MaxFinder(int[] array, int start, int end, int idHilo) {
            this.array = array;
            this.start = start;
            this.end = end;
            this.idHilo = idHilo;
        }

        // Declaramos el metodo getter que devuelve el maximo encontrado por este hilo
        public int getLocalMax() {
            return localMax;
        }

        @Override
        // Metodo run que utilizamos para recorrer el segmento asignado del array y asi obtener el numero maximo que hay.
        public void run() {
            localMax = array[start];
            for (int i = start + 1; i < end; i++) {
                if (array[i] > localMax) {
                    localMax = array[i];
                }
            }

            // Mostramos el numero máximo encontrado por el hilo
            System.out.println("Hilo " + idHilo + " -> Máximo encontrado: " + localMax);
        }
    }

    // Metodo principal de la funcion
    public static void main(String[] args) throws InterruptedException {

        // Declaramos el tamaño del array
        int NUM_DIAS = 3650;
        // Declaramos la variable con el NUM_HILOS que indica en cuantos hilos lo dividimos
        int NUM_HILOS = 4;

        // Generar el array con temperaturas aleatorias entre -30 y 50 grados
        int[] temperaturas = new int[NUM_DIAS];
        Random r = new Random();

        for (int i = 0; i < NUM_DIAS; i++) {
            temperaturas[i] = r.nextInt(81) - 30;
        }

        // Dividir el array en partes iguales para cada hilo
        MaxFinder[] hilos = new MaxFinder[NUM_HILOS];
        int tamañoParte = NUM_DIAS / NUM_HILOS;

        for (int i = 0; i < NUM_HILOS; i++) {
            // Aqui marcamos la posicion inicial del segmento que procesará este hilo
            int start = i * tamañoParte;
            // Aqui se marca la posición final del segmento que procesara este hilo
            int end = (i + 1) * tamañoParte;
            hilos[i] = new MaxFinder(temperaturas, start, end, i);
            hilos[i].start();
        }

        // Esperamos a que todos los hilos terminen
        for (MaxFinder hilo : hilos) {
            hilo.join();
        }

        // Mostramos el resultado final
        int maximo = 0;
        for (MaxFinder hilo : hilos) {
            if (hilo.getLocalMax() > maximo) {
                maximo = hilo.getLocalMax();
            }
        }

        System.out.println("Temperatura máxima encontrada: " + maximo);
    }
}
```

```

int end = (i == NUM_HILOS - 1) ? NUM_DIAS : start + tamañoParte;
// creamos el hilo encargado de procesar el segmento entre las posiciones del start y el end
hilos[i] = new MaxFinder(temperaturas, start, end, i + 1);
// Ejecutamos el hilo
hilos[i].start();
}

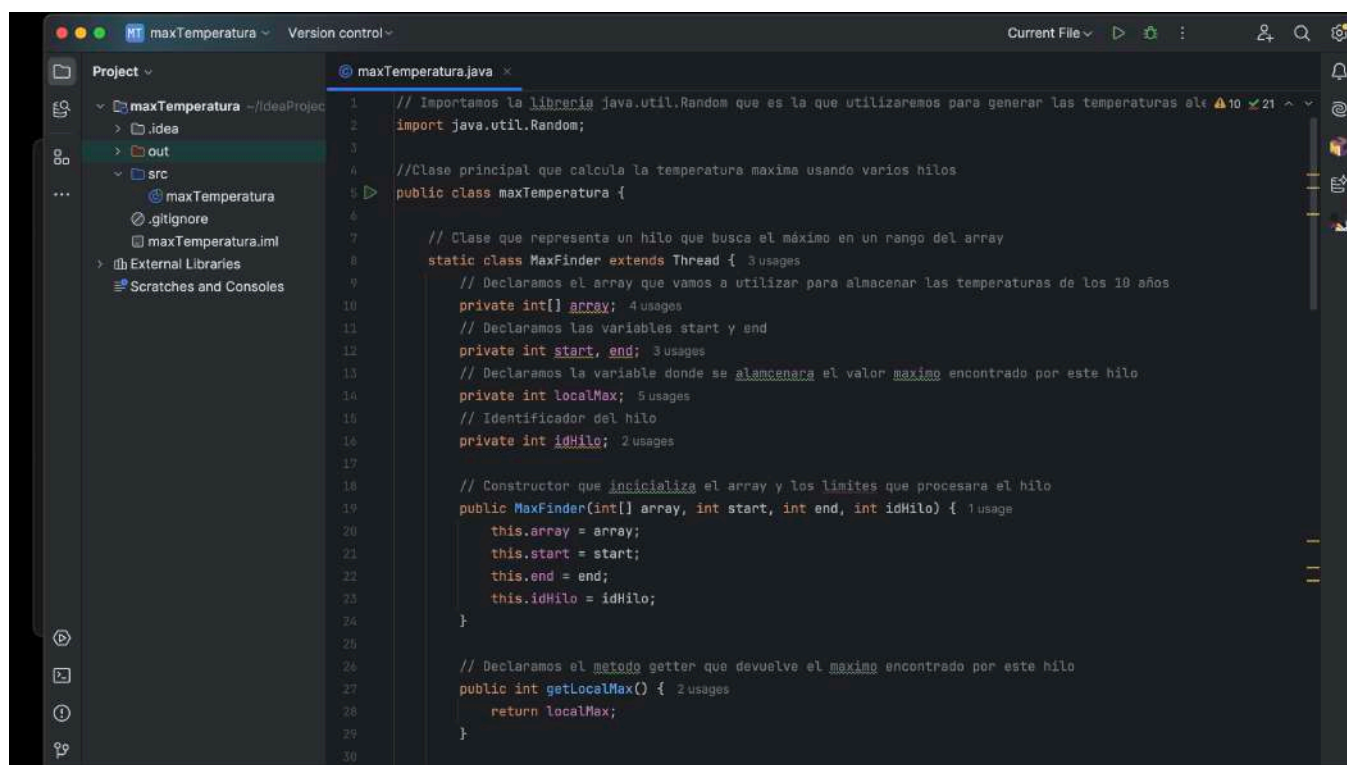
// Espera a que todos los hilos terminen y calcula el máximo global
// Variable donde almacenaremos el máximo global ( inicializada al mínimo entero posible)
int maximaGlobal = Integer.MIN_VALUE;

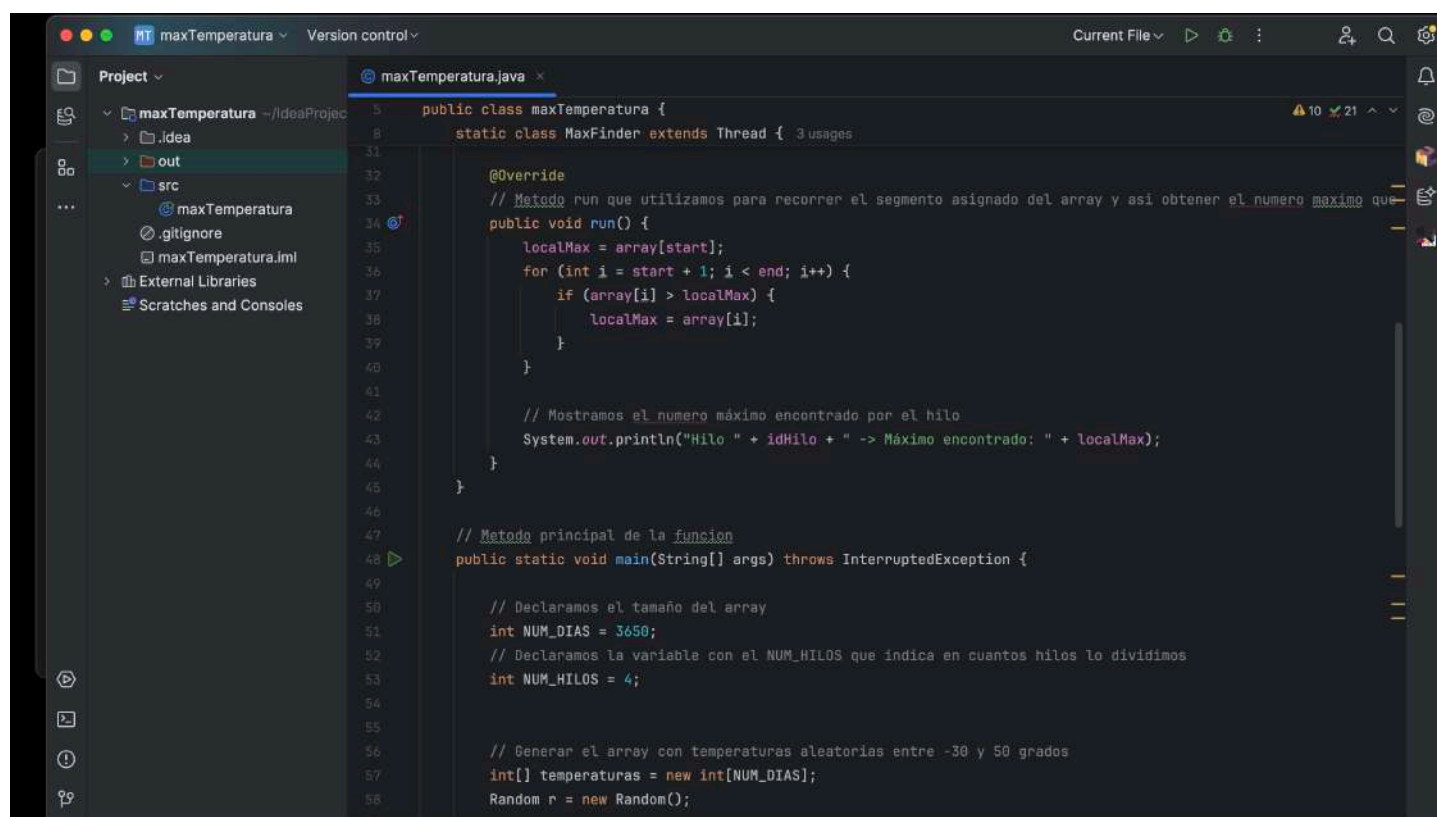
// Recorremos los diferentes hilos, todos, y miramos a ver cual es el valor maximo de todos
for (int i = 0; i < NUM_HILOS; i++) {
    hilos[i].join();
    if (hilos[i].getLocalMax() > maximaGlobal) {
        maximaGlobal = hilos[i].getLocalMax();
    }
}

// Mostrar el resultado final con la temperatura maxima de todos los hilos
System.out.println("\nTemperatura maxima encontrada por todos los hilos: " + maximaGlobal);
}
}

```

Muestro capturas de pantalla para ver el código en el IDE

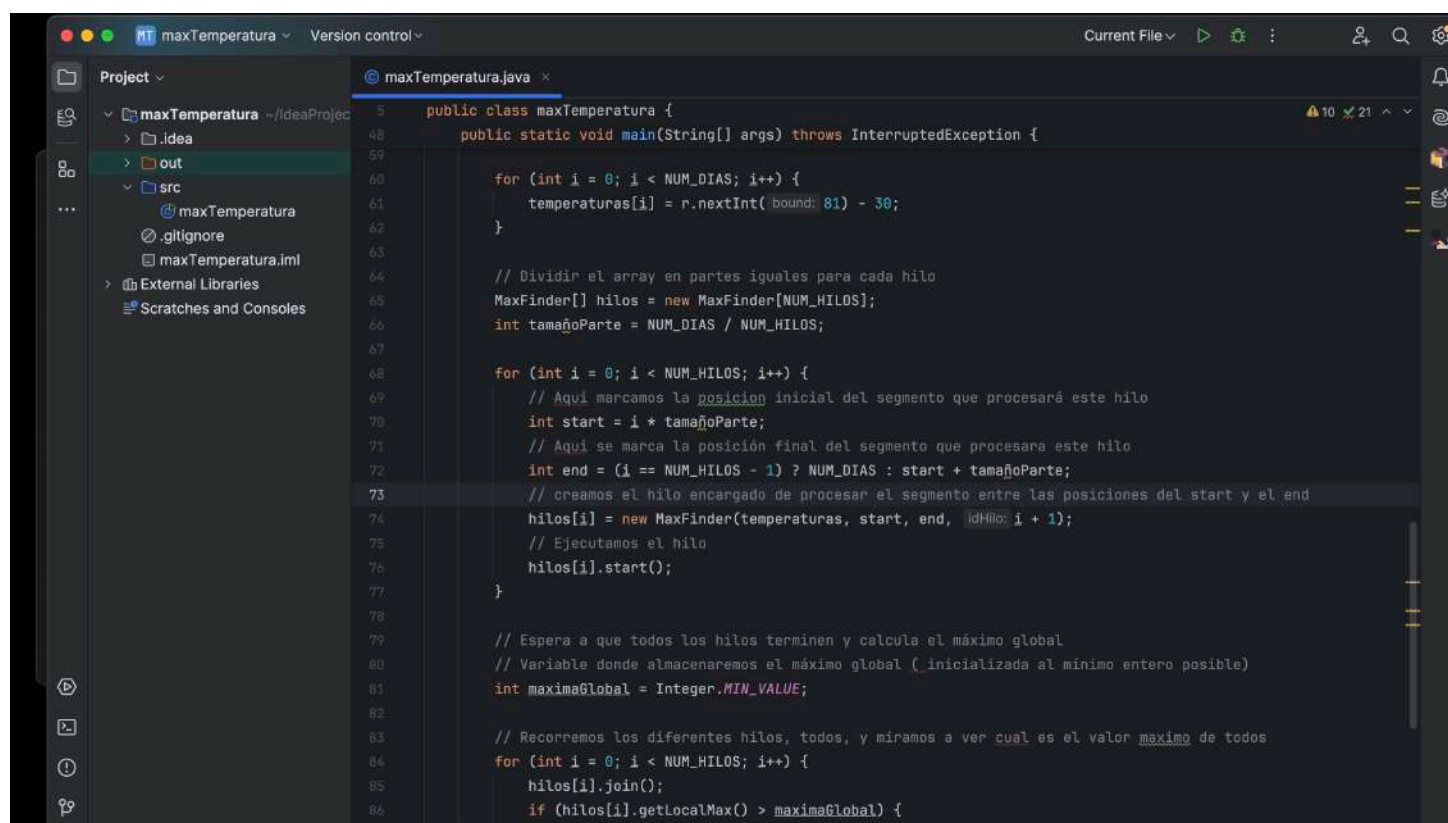




```

1  public class maxTemperatura {
2      static class MaxFinder extends Thread {
3          // 3 usages
4
5          @Override
6          // Metodo run que utilizamos para recorrer el segmento asignado del array y asi obtener el numero maximo que
7          public void run() {
8              localMax = array[start];
9              for (int i = start + 1; i < end; i++) {
10                 if (array[i] > localMax) {
11                     localMax = array[i];
12                 }
13             }
14
15             // Mostramos el numero máximo encontrado por el hilo
16             System.out.println("Hilo " + idHilo + " -> Máximo encontrado: " + localMax);
17         }
18     }
19
20     // Metodo principal de la funcion
21     public static void main(String[] args) throws InterruptedException {
22
23         // Declaramos el tamaño del array
24         int NUM_DIAS = 3650;
25         // Declaramos la variable con el NUM_HILOS que indica en cuantos hilos lo dividimos
26         int NUM_HILOS = 4;
27
28         // Generar el array con temperaturas aleatorias entre -30 y 50 grados
29         int[] temperaturas = new int[NUM_DIAS];
30         Random r = new Random();
31     }
32 }

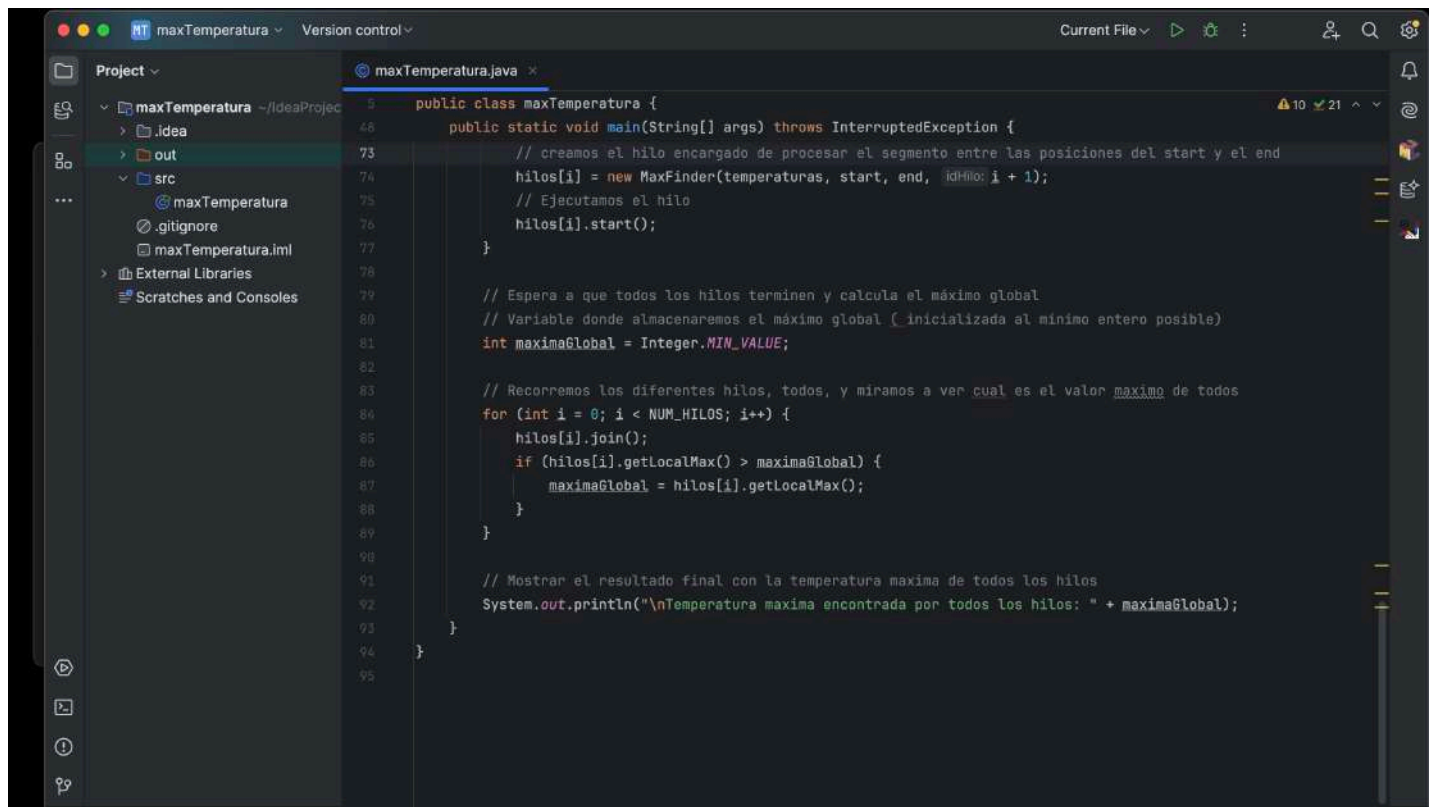
```



```

33     for (int i = 0; i < NUM_DIAS; i++) {
34         temperaturas[i] = r.nextInt(-bound: 81) - 30;
35     }
36
37     // Dividir el array en partes iguales para cada hilo
38     MaxFinder[] hilos = new MaxFinder[NUM_HILOS];
39     int tamañoParte = NUM_DIAS / NUM_HILOS;
40
41     for (int i = 0; i < NUM_HILOS; i++) {
42         // Aquí marcamos la posición inicial del segmento que procesará este hilo
43         int start = i * tamañoParte;
44         // Aquí se marca la posición final del segmento que procesara este hilo
45         int end = (i == NUM_HILOS - 1) ? NUM_DIAS : start + tamañoParte;
46         // creamos el hilo encargado de procesar el segmento entre las posiciones del start y el end
47         hilos[i] = new MaxFinder(temperaturas, start, end, idHilo: i + 1);
48         // Ejecutamos el hilo
49         hilos[i].start();
50     }
51
52     // Espera a que todos los hilos terminen y calcula el máximo global
53     // Variable donde almacenaremos el máximo global ( inicializada al mínimo entero posible)
54     int maximaGlobal = Integer.MIN_VALUE;
55
56     // Recorremos los diferentes hilos, todos, y miramos a ver cual es el valor maximo de todos
57     for (int i = 0; i < NUM_HILOS; i++) {
58         hilos[i].join();
59         if (hilos[i].getLocalMax() > maximaGlobal) {
60
61         }
62     }
63 }

```

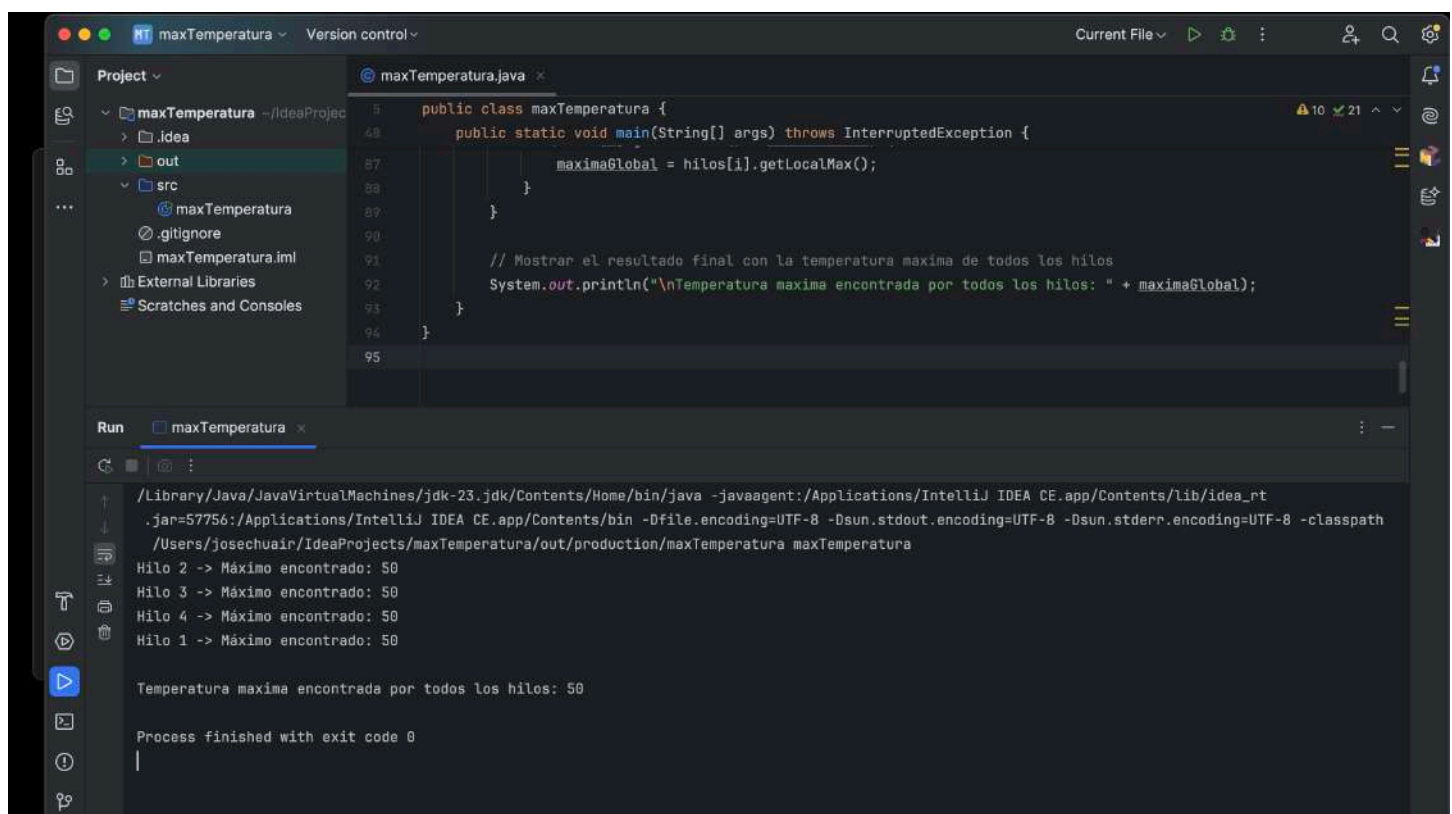


```

1  public class maxTemperatura {
2
3      public static void main(String[] args) throws InterruptedException {
4
5          // creamos el hilo encargado de procesar el segmento entre las posiciones del start y el end
6          hilos[i] = new MaxFinder(temperaturas, start, end, idHilo: i + 1);
7          // Ejecutamos el hilo
8          hilos[i].start();
9      }
10
11      // Espera a que todos los hilos terminen y calcula el máximo global
12      // Variable donde almacenaremos el máximo global ( inicializada al minimo entero posible)
13      int maximaGlobal = Integer.MIN_VALUE;
14
15      // Recorremos los diferentes hilos, todos, y miramos a ver cual es el valor maximo de todos
16      for (int i = 0; i < NUM_HILOS; i++) {
17          hilos[i].join();
18          if (hilos[i].getLocalMax() > maximaGlobal) {
19              maximaGlobal = hilos[i].getLocalMax();
20          }
21      }
22
23      // Mostrar el resultado final con la temperatura maxima de todos los hilos
24      System.out.println("\nTemperatura maxima encontrada por todos los hilos: " + maximaGlobal);
25  }
26  }

```

Y una última captura mostrando el resultado :



```

/Library/Java/JavaVirtualMachines/jdk-23.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib/idea_rt
.jar=57756:/Applications/IntelliJ IDEA CE.app/Contents/bin -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath
/Users/josechuair/IdeaProjects/maxTemperatura/out/production/maxTemperatura maxTemperatura
Hilo 2 -> Máximo encontrado: 50
Hilo 3 -> Máximo encontrado: 50
Hilo 4 -> Máximo encontrado: 50
Hilo 1 -> Máximo encontrado: 50

Temperatura maxima encontrada por todos los hilos: 50

Process finished with exit code 0

```